

Assignment 3 :

1. GitHub Link :

https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/tree/main/DAY2/Assignment%203

2. README.md Link :

https://github.com/ssbagi/IBM_OpenPOWER_and_SoC_Design/blob/main/DAY2/Assignment%203/README.md

Question 1 : Design and Simulate 4-bit BCD to Excess-3 code converter using Verilog HDL and verify its operation through simulation.

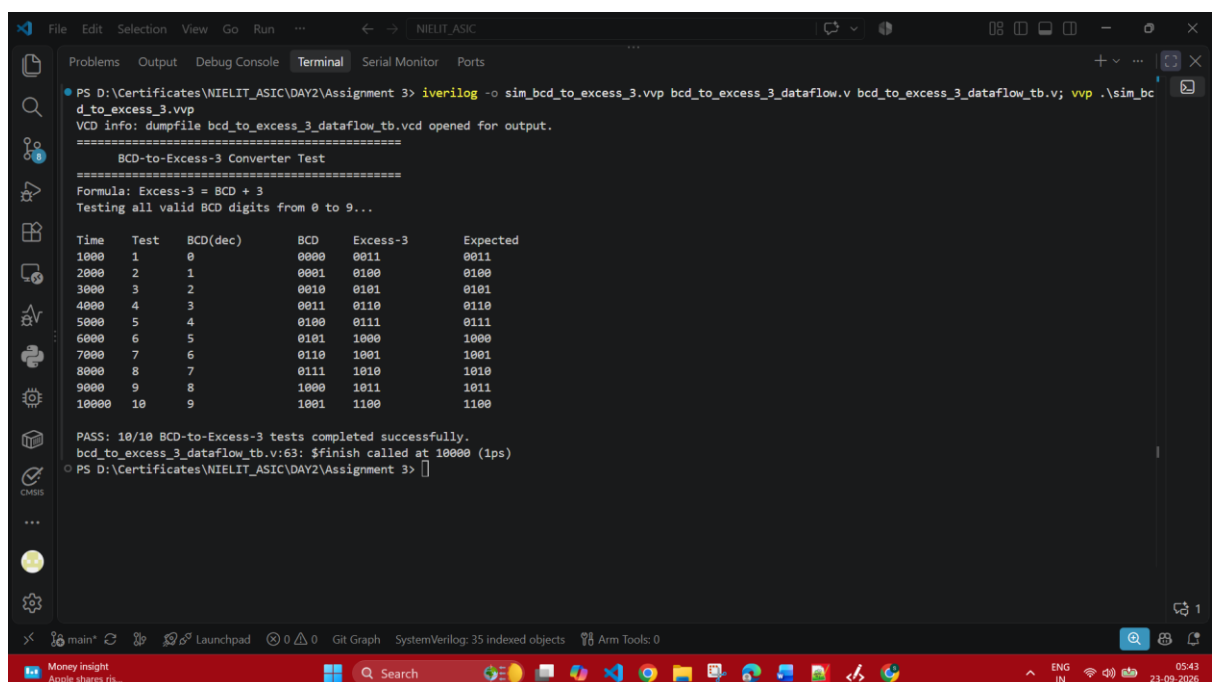
Verilog code :

```
module bcd_to_excess_3_dataflow (
    input [3:0] bcd,
    output [3:0] excess_3
);

    assign excess_3 = bcd + 4'b0011;

endmodule
```

Compilation :



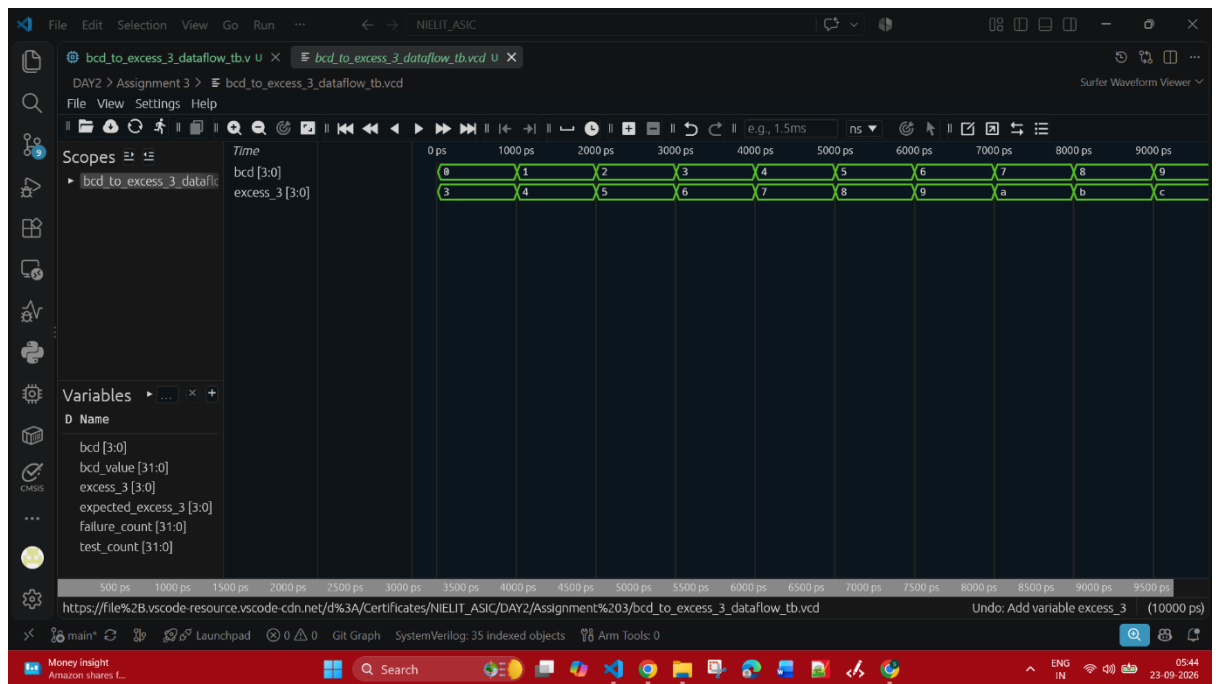
The screenshot shows a terminal window with the following output:

```
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 3> iverilog -o sim_bcd_to_excess_3.vvp bcd_to_excess_3_dataflow.v bcd_to_excess_3_dataflow_tb.v; vvp .\sim_bcd_to_excess_3.vvp
VCD Info: dumpfile bcd_to_excess_3_dataflow_tb.vcd opened for output.
=====
BCD-to-Excess-3 Converter Test
=====
Formula: Excess-3 = BCD + 3
Testing all valid BCD digits from 0 to 9...

Time    Test    BCD(dec)    BCD    Excess-3    Expected
1000    1         0          0000    0011        0011
2000    2         1          0001    0100        0100
3000    3         2          0010    0101        0101
4000    4         3          0011    0110        0110
5000    5         4          0100    0111        0111
6000    6         5          0101    1000        1000
7000    7         6          0110    1001        1001
8000    8         7          0111    1010        1010
9000    9         8          1000    1011        1011
10000   10        9          1001    1100        1100

PASS: 10/10 BCD-to-Excess-3 tests completed successfully.
bcd_to_excess_3_dataflow_tb.v:63: $finish called at 10000 (1ps)
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 3>
```

Simulation :



Question 2 : Design and simulate a Verilog module that converts a 4-bit BCD input into the corresponding seven segment display outputs.

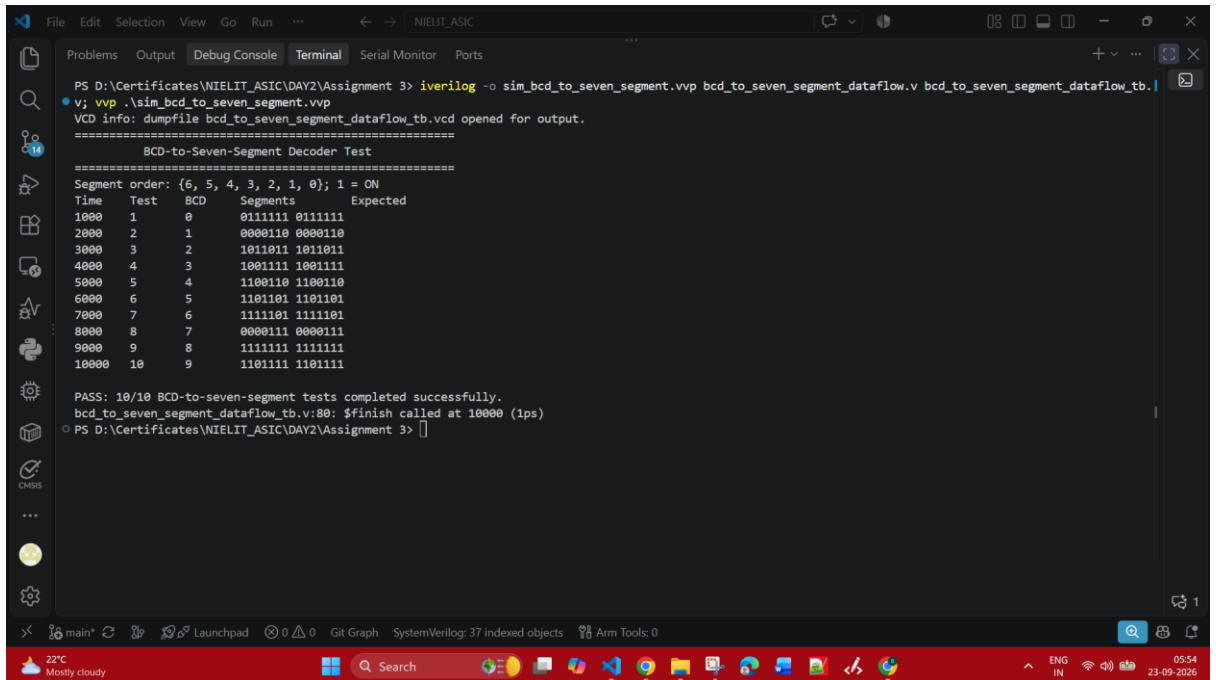
Verilog code :

```
module bcd_to_seven_segment_dataflow (  
    input [3:0] bcd,  
    output reg [6:0] segment  
);
```

```
    always @(*) begin  
        case (bcd)  
            4'd0: segment = 7'b0111111;  
            4'd1: segment = 7'b0000110;  
            4'd2: segment = 7'b1011011;  
            4'd3: segment = 7'b1001111;  
            4'd4: segment = 7'b1100110;  
            4'd5: segment = 7'b1101101;  
            4'd6: segment = 7'b1111101;  
            4'd7: segment = 7'b0000111;  
            4'd8: segment = 7'b1111111;  
            4'd9: segment = 7'b1101111;  
            default: segment = 7'b0000000;  
        endcase  
    end
```

```
endmodule
```

Compilation :



```
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 3> iverilog -o sim_bcd_to_seven_segment.vvp bcd_to_seven_segment_dataflow.v bcd_to_seven_segment_dataflow_tb.v
v; vvp .\sim_bcd_to_seven_segment.vvp
VCD info: dumpfile bcd_to_seven_segment_dataflow_tb.vcd opened for output.
=====
          BCD-to-Seven-Segment Decoder Test
=====
Segment order: {6, 5, 4, 3, 2, 1, 0}; 1 = ON
Time   Test   BCD   Segments   Expected
1000    1       0     0111111   0111111
2000    2       1     0000110   0000110
3000    3       2     1011011   1011011
4000    4       3     1001111   1001111
5000    5       4     1100110   1100110
6000    6       5     1101101   1101101
7000    7       6     1111101   1111101
8000    8       7     0000111   0000111
9000    9       8     1111111   1111111
10000   10      9     1101111   1101111

PASS: 10/10 BCD-to-seven-segment tests completed successfully.
bcd_to_seven_segment_dataflow_tb.v:80: $finish called at 10000 (1ps)
PS D:\Certificates\NIELIT_ASIC\DAY2\Assignment 3>
```

Simulation :

